Introduction to computer programming (2023/2024)

Paolo Franciosa (credits to Massimo Lauria)

Exam December 15, 2023

Working methods: The exam text, the test files of the exercises and a synoptic table on Python syntax can be found in the `exam` folder on your desk, in the "Statistica-Laboratorio" virtual machine. The solutions to the exercises must be placed in the same folder `exam`. **First compile the file** `studente.txt` with the first name, surname and matriculation information

```
nome: Giacomo
cognome: Leopardi
matricola: 123456789
```

Not completing the `student.txt` file is like not handing in the assignment. For the exam, you can only have

- your ID or passport;
- beverages and snacks;

and you must leave your bags and backpacks in the corner of the room, with mobile phones and electronic devices switched off inside.

The grade is the sum of the points of the exercises, so it is possible to get 30 even without completing them all.

Use of test files: tests check very briefly the setting of the solutions and passing the tests does not at all imply that the solution is correct. Read the text of the exercises before starting to solve them. To complete each exercise exercise you must

- write a solution file **with the specified name**;
- have the function **with the specified name** inside;
- place the solution file in the `exam` folder;

- run the `python3 <testfile>` command in that folder

where `<testfile>` should of course be replaced with the name of the appropriate test file for the exercise you are working on.

There is an independent test file for each exercise, so you can work on the exercises one at a time with more ease. The result of each test is a screen (or several screens) showing

- whether the file with the correct name was found,
- whether the file contains the function with the correct name,
- whether calls to the function with different parameter values end by returning correct results.

For each function written, calls are made with different values of the parameters. The result of the tests is returned with the character

- `E` if the call cannot be executed,
- `F` if the function does not return the correct result,
- `.` if the function returns the correct result.

1 Circle area

Write a function `circle_area(x)` that:

- takes a non-negative number `x` as parameter;
- returns the area of a circle with radius `x`.

After importing `math` package, a good approximation of Pi is available as `math.pi`

The Python function must be saved in file: `circle_area.py`

Test file: `test_circle_area.py`

2 Average

Write a function `average(L)` that,

- takes a list `L` as parameter, in which each element is a number (`int` or `float`);
- returns the average of asll numbers in the list `L`. You can assume that `L` is a list of numbers (each element is `int` oppure `float`).

The Python function must be saved in file: `average.py`

Test file: `test_average.py`

3 Scalar product between two vectors

The scalar product between two vectors $\langle a_0, a_1, \ldots, a_{n-1} \rangle$ and $\langle b_0, b_1, \ldots, b_{n-1} \rangle$ is defined as

$$\sum_{i=0}^{n-1} a_i b_i$$

We represent vectors as lists of numbers.

Write a function `scalar_product` that:

- takes two non-empty lists `a`, `b` with the same length as parameters, where each element is a number (`int` or `float`);
- returns the scalar product of the two lists.

The scalar product between two lists is therefore the sum of the products of the homologous elements. For example, the scalar product between the vectors `[1, 4, 3]` and `[5, 5, 1]` is $1 \times 5 + 4 \times 5 + 3 \times 1 = 28$. The scalar product of two vectors is a numerical value.

You can **assume** that the lists are not empty, that they contain only numbers and that they have the same length.

The Python function must be saved in file: `scalar_product.py`

Test file: `test_scalar_product.py`

4 Check that all values in a dictionary are small

Write a function `dict_check_small_values(d, threshold)` that:

- takes as arguments a dictionary `d` with values of type `int` or `float` and an number `threshold`;
- returns `True` if all values are strictly smaller than `threshold`, and returns `False` otherwise.

You may assume that alle values are numeric.

For example:

```
d = {"Mario": 1000, "Gianna" : 8000, "Alberto": 4000}

print(dict_check_small_values(d, 7000))
```

```
False
```

The Python function must be saved in file: `dict_check_small_values.py`

Test file: `test_dict_check_small_values.py`

5 Count large elements in a matrix

Write a function `mat_count_large(m, threshold)` that:

- takes as argument a matrix `m`, represented as a list of lists;
- returns the number of elements in `m` that are strictly larger than `threshold`.

We represent an $r \times c$ matrix in Python as a list with length r, where each element is a list of length c. You may assume that the argument is well formed, with $r \geq 1$, $c \geq 1$, and all the lists representing rows contain `c` numeric elements.

The Python function must be saved in file: `mat_count_large.py`

Test file: `test_mat_count_large.py`

6 Change minima to zero

Write a function `change_mins_in_zero(values)` that:

- expects a non-empty list of comparable values as an argument;
- directly modifies the `values` list by replacing the minimum values with zero.

If the minimum appears several times in the list, the function must replace all occurrences of the minimum value with zero.

The Python function must be saved in file: `change_mins_in_zero.py`

Test file: `test_change_mins_in_zero.py`